

Workflow-Centered Evaluation for Long-Horizon Agent Workflows

****Status:**** final revised conceptual/methods paper draft with illustrative mini pilot ****Date:**** 2026-05-02

****Theme:**** evaluation-benchmarking

Abstract

Evaluation practice for language-model agents still tends to privilege the base model as the primary explanatory unit of performance. That assumption becomes increasingly unstable in long-horizon, tool-using, budget-constrained settings, where observed outcomes depend not only on the model, but also on workflow structure, environmental affordances, and resource limits. This paper argues that contemporary agent evaluation therefore suffers from a unit mismatch: practical systems are often discussed as if they were model comparisons, while they are in fact workflow-conditioned system comparisons. We propose a workflow-centered evaluation framework in which performance is treated schematically as a function of ****model****, ****workflow****, ****environment****, and ****budget****. We define five evaluation dimensions—task quality, cost efficiency, recoverability, auditability, and robustness—and operationalize recoverability and auditability as provisional proxy metrics suitable for first-pass empirical study. We situate the proposal within recent benchmark literature on planning, tool use, interactive environments, software-engineering agents, and reflective control. We then introduce a factorized protocol for separating workflow effects from model effects and report a small executable mini pilot on repository-repair tasks. The pilot is intentionally limited: it does not support the paper’s central workflow-dominance hypothesis, but it does show that additional workflow structure can increase trace richness and token cost without improving success on easy tasks. The paper’s contribution is therefore methodological rather than demonstrative. Its aim is to define a more faithful object of measurement and a falsifiable research program for evaluating long-horizon agent workflows.

1. Introduction

Benchmark-centered evaluation remains one of the dominant habits of contemporary AI discourse. A model is run on a task suite, a scalar score is obtained, and that score is treated as evidence about capability. This practice remains useful in many contexts. Benchmarks can expose regressions, support local comparison, and reveal narrow strengths and weaknesses. However, for long-horizon, tool-using, stateful agent workflows, model-centered evaluation increasingly obscures the object of interest.

The problem is not merely that existing benchmarks are incomplete, noisy, or susceptible to overfitting. The deeper issue is ontological: benchmark outputs are often interpreted as if they measured the model in isolation, even when practical system behavior depends materially on prompt scaffold, memory policy, retrieval design, decomposition strategy, retry logic, checkpointing, tool-calling structure, and trace persistence. In many real workflows, these are not incidental implementation details. They influence whether a system completes the task, recovers from perturbation, remains inspectable, and stays within operational cost bounds.

As a consequence, benchmark scores may conceal rather than clarify causal structure. Two systems that appear similar on a leaderboard may differ substantially in completion stability, recovery behavior, latency, cost, and trace legibility. Conversely, systems built on different models may converge in practical quality when workflow structure compensates for model differences. This creates a recurrent interpretive error: model-centered language is used to describe outcomes that are, in operational terms, workflow-conditioned.

This paper advances a deliberately narrow claim. It does not argue that all agent evaluation should be replaced by workflow-centered evaluation, nor that model comparison has ceased to matter. It argues only that **for long-horizon, tool-using, budget-constrained workflows**, evaluation should shift from model-centered measurement toward workflow-centered measurement. In this regime, treating workflow as nuisance variance risks standardizing away mechanisms that actual users depend on.

1.1 Contributions

This paper makes six contributions.

1. It identifies a **unit mismatch** in current agent evaluation: the system being measured is often richer than the model being discussed.
2. It proposes a compact **schematic decomposition** of observed performance into model, workflow, environment, and budget.
3. It defines five workflow-relevant evaluation dimensions: **task quality**, **cost efficiency**, **recoverability**, **auditability**, and **robustness**.
4. It operationalizes **recoverability** and **auditability** as provisional proxy metrics suitable for first-pass empirical study.
5. It outlines a **factorized experimental protocol** for comparing workflow effects against model effects.
6. It reports a **small executable mini pilot** whose primary value is methodological and disciplinary: it shows how workflow instrumentation can be measured, while also showing that stronger workflow structure is not automatically beneficial on easy tasks.

The paper is primarily conceptual and methodological. Its empirical component is intentionally illustrative and should not be interpreted as a definitive test of the central hypothesis.

2. Related Work

Recent literature already reveals stress points in model-centered evaluation, but does so from several different directions.

2.1 Diagnostic reasoning and planning benchmarks

Planning-focused work such as **PlanBench** (Valmeekam et al., 2023) argues that many common-sense planning tasks are poor instruments for distinguishing genuine planning from retrieval or pattern completion. Its main relevance here is methodological: benchmark design can quietly alter what capability is actually being measured. Likewise, **ActionReasoningBench** (Handa et al., 2025) probes reasoning about actions, state tracking, executability, and ramifications. Such benchmarks remain valuable for local diagnosis, but they do not attempt to characterize recoverability, auditability, or budgeted long-horizon behavior.

2.2 Interactive and environment-rich agent benchmarks

A second line of work moves closer to realistic agent settings by embedding models in interactive environments. **AgentBench** (Liu et al., 2024) evaluates agents across eight interactive environments and explicitly frames the need to evaluate LLMs as agents rather than as static answer generators. **GAIA** (Mialon et al., 2023) similarly emphasizes tool use, web access, and multimodal real-world problem solving. **OSWorld** (Xie et al., 2024) extends this realism by providing execution-based evaluation in real computer environments across operating systems, while **VisualWebArena** (Koh et al., 2024) evaluates multimodal agents on visually grounded web tasks.

These benchmarks are highly relevant because they recognize that useful agent behavior emerges through action within environments rather than isolated response generation. However, even when environmental realism improves, the principal output often remains a task success metric, leaving workflow contributions comparatively under-theorized.

2.3 Software-engineering and domain-specific agent benchmarks

SWE-bench (Jimenez et al., 2024) is especially important because it operationalizes long-context, repository-level software repair using real GitHub issues and executable validation. It demonstrates that realistic software-engineering tasks require coordination across files, interaction with execution environments, and more than local code completion. Likewise, **MIRAI** (Ye et al., 2024) evaluates forecasting agents that must gather information, use tools through APIs, and reason across temporal data.

These benchmarks greatly widen the task surface, yet they still leave open a structural question: when an agent succeeds or fails, how much of that outcome should be attributed to the base model and how much to workflow design?

2.4 Workflow interventions and reflective control

A fourth line of work studies workflow interventions more directly. **Devil's Advocate** (Wang et al., 2024) adds anticipatory reflection, remedy planning, and review loops for web agents, reporting better performance and fewer trials in WebArena-style settings. The significance of such work is not merely that a particular intervention helps. Rather, it demonstrates that workflow changes—planning, reflection, backtracking, and repair loops—can materially alter outcomes even when the base model is held fixed. Once that is true, workflow cannot be treated purely as background implementation variance.

2.5 Synthesis and position of the present paper

Taken together, existing work is increasingly strong on two fronts: **environmental realism** and **local capability diagnosis**. It is comparatively weaker on a third question: how to decompose workflow contributions as first-class explanatory variables when interpreting system performance. The present paper does not propose a new benchmark or a new agent architecture. Its contribution is methodological: it argues that, for a specific regime of long-horizon, tool-using, budget-constrained tasks, the explanatory unit of evaluation should widen from the model to the workflow-conditioned system.

3. The Unit Mismatch Problem

The central claim of this paper is that current agent evaluation often suffers from a mismatch between the **measured unit** and the **discussed unit**.

In standard benchmark discourse, the discussed unit is usually the model. A benchmark score is interpreted as evidence about what the model can do. Prompting, tool wrappers, or orchestration layers may be acknowledged, but they are typically treated as secondary context.

In long-horizon deployed workflows, however, the measured unit is rarely the model in isolation. The acting system is ordinarily composed of at least:

- a base language model,
- a workflow that structures action and state transition,
- an environment with specific affordances and failure modes,
- a budget regime that constrains feasible action.

If these components jointly determine practical outcomes, then single-score comparisons become epistemically unstable. A reported score may partly reflect model strength, but it may also reflect

checkpointing policy, retrieval design, retry discipline, tool ergonomics, or evaluator-specific asymmetries. When these terms are not separated analytically, benchmark outcomes are easy to overinterpret.

This problem becomes especially acute under four conditions:

1. **Long horizons**, where small workflow differences accumulate.
2. **Tool dependence**, where success depends on interaction quality rather than text generation alone.
3. **Budget constraints**, where token, latency, and retry ceilings shape feasible behavior.
4. **Audit or oversight requirements**, where the inspectability of intermediate traces matters independently of final correctness.

The claim here is conceptual, not yet empirical: benchmark scores are not meaningless, but they are often used to answer causal questions they were not designed to resolve.

4. A Schematic Decomposition of Performance

We propose the following schematic decomposition:

$$P = f(M, W, E, B)$$

where:

- P denotes observed performance,
- M denotes the base model,
- W denotes workflow,
- E denotes environment,
- B denotes budget constraints.

This expression is **not** offered as a complete mathematical theory of agent performance. It is a schematic decomposition intended to force explicit recognition of variables that practical evaluation often collapses.

4.1 Model (M)

The model term includes the base language model and any directly model-specific inference behavior relevant to the run.

4.2 Workflow (W)

Workflow denotes the structured procedure through which the model acts: prompt scaffold, memory policy, decomposition logic, retry rules, checkpointing, tool use, and trace persistence.

4.3 Environment (E)

Environment includes the task substrate and interface ecology: APIs, file systems, websites, execution sandboxes, document formats, and interface brittleness.

4.4 Budget (B)

Budget includes token ceilings, latency limits, retry caps, tool-use quotas, and monetary constraints.

4.5 Interaction terms

These variables are not independent. Workflow may interact nonlinearly with budget; environment may amplify or suppress workflow advantages; model effects may grow or shrink depending on workflow structure. The usefulness of the decomposition lies not in assumed separability, but in experimental discipline: it specifies which variables should be manipulated or held fixed when interpreting performance.

5. Taxonomy of Workflow Components

For workflow-centered evaluation to be analytically useful, “workflow” must not remain a vague residual category. We therefore distinguish six components.

1. **Prompt scaffold**: system instructions, turn templates, continuation logic, and explicit planning structure.
2. **Memory policy**: what is persisted, summarized, retrieved, compressed, or discarded.
3. **Decomposition strategy**: how tasks are partitioned into subgoals and whether intermediate verification occurs.
4. **Retry and recovery logic**: how failures are classified, repaired, retried, or resumed.
5. **Tool-calling structure**: how tools are selected, invoked, chained, and verified.
6. **Trace persistence and audit layer**: what intermediate artifacts are stored and whether they remain legible.

These components are analytic handles rather than perfectly orthogonal subsystems. Their purpose is to support disciplined comparison.

6. Evaluation Dimensions and Proxy Metrics

If workflow is part of the object of evaluation, then the measured dimensions must widen accordingly.

6.1 Task quality

Did the system solve the task correctly, satisfy constraints, and produce a usable artifact?

6.2 Cost efficiency

What token, latency, tool-use, retry, and human-review budget was required?

6.3 Recoverability

Recoverability is the system's ability to resume or repair after perturbation or local failure without full task restart.

We propose the following **provisional proxy metric**:

$$R = \frac{\text{successful recoveries}}{\text{recoverable failure events}}$$

A recoverable failure event is any failure that leaves continuation possible in principle: for example tool timeout, malformed retrieval result, interrupted substep, or broken local plan.

Useful secondary measurements include:

- mean recovery latency,
- additional recovery cost,
- preservation of trace continuity during recovery.

6.4 Auditability

Auditability is the extent to which a competent evaluator can reconstruct what the agent did, why it did it, and where failure entered the workflow.

We propose the following **provisional proxy metric**:

$$A = \frac{V + S + Q}{3}$$

where:

- V is **step visibility**,
- S is **state continuity**,

- $\mathcal{Q}$ is **reason-trace adequacy**,

and each component is normalized to $[0,1]$.

6.5 Robustness

How much does performance degrade under paraphrase, noisy inputs, tool instability, altered document formats, or sequencing changes?

6.6 Metric caveat

The claims in this section are methodological proposals, not validated measurement conclusions. The recoverability and auditability metrics are intentionally incomplete. Both are partly evaluator-dependent, both omit potentially important qualitative distinctions, and neither should be mistaken for a final metric theory. Their function is narrower: to make workflow properties instrumentable enough for comparative study.

These dimensions should not be collapsed too quickly into a single scalar leaderboard. Different deployment settings may legitimately weight them differently.

7. Factorized Experimental Protocol

The core empirical design implied by the framework is simple:

1. **Hold the model fixed; vary the workflow.**
2. **Hold the workflow fixed; vary the model.**

This creates a minimum design for separating workflow effects from model effects.

7.1 Suitable task families

The protocol is most relevant for task classes where workflow plausibly matters:

- long-horizon coding and repository modification,
- multi-step research synthesis,
- document-grounded analysis with retrieval,
- tool-using administrative workflows,
- resumable tasks with interruptions,
- explicitly budgeted tasks.

7.2 Core measurements

For each run, record at minimum:

- final task quality,
- total token cost,
- latency,
- tool calls,
- failure incidence,
- recovery success rate,
- trace completeness,
- perturbation sensitivity.

These data should be stored at the run level rather than only as aggregate scores, because workflow failures are typically heterogeneous.

8. Worked Protocol Sketch

To make the framework concrete, consider a repository-modification stress-test domain in which an agent must:

1. inspect a codebase,
2. identify target files,
3. implement a requested change,
4. validate or reason about correctness,
5. preserve a legible trace.

A fixed-model comparison could use four workflows:

- **W1**: direct execution, no persistent memory, no checkpointing;
- **W2**: retrieval memory only;
- **W3**: retrieval memory plus explicit checkpoints;
- **W4**: retrieval memory, checkpoints, and bounded repair loops.

Inject one perturbation per task instance, such as command failure, missing path, noisy retrieval, reduced token budget, or interrupted execution. Measure quality, cost, recoverability, auditability, and robustness loss relative to a clean-path baseline.

This task family should not be mistaken for a universal proxy for all agent work. It is a **stress-test domain** chosen precisely because workflow should matter there. If workflow differences fail to appear even under such conditions, the present thesis weakens considerably.

9. Illustrative Mini Pilot

The preceding sections develop a conceptual framework and a proposed measurement program. The remainder of the paper reports a small empirical illustration. Its purpose is limited: it is not meant to establish the paper’s central hypothesis, but to test whether the proposed framework can support a cheap, executable, and reproducible workflow comparison.

9.1 Pilot methods

The pilot used `gpt-4.1` through the OpenAI Chat Completions API on **2026-05-02**. A custom harness (`workflow_eval_pilot.py`) instantiated four small Python repository-repair tasks with executable `unittest` validation. Each task contained a bug or required coordinated local edits, and success was defined by post-edit test passage.

Two workflows were compared:

- **Direct**: single-pass edit generation;
- **Plan-then-edit**: explicit repair-plan generation followed by edit generation with a self-check field.

Both workflows received the same repository files and the same task instruction. The harness used deterministic prompting with temperature set to zero, parsed structured JSON outputs, applied returned file edits in a temporary workspace, and executed tests locally. Measured quantities in this pilot were task success and total token usage. “Trace richness” was not given a formal scalar score here; instead, it refers narrowly to the presence of explicit intermediate plan and self-check artifacts in the structured workflow outputs. The code and raw results are stored locally alongside the draft.

9.2 Pilot tasks

The four tasks were:

1. `off_by_one_sum`
2. `csv_parser_whitespace`
3. `discount_cap`
4. `dependent_update`

These tasks are intentionally small. They therefore function only as a feasibility probe, not as a serious challenge benchmark.

9.3 Pilot results

Workflow	Pass rate	Mean total tokens	Explicit intermediate artifacts
Direct	4/4	279.75	Minimal
Plan-then-edit	4/4	781.5	Present

The structured workflow consumed approximately $2.79\times$ as many total tokens as the direct workflow while yielding no improvement in task success on these four easy tasks.

9.4 What the pilot does and does not show

The pilot shows three things.

First, workflow instrumentation can be measured reproducibly in a lightweight executable harness. Second, stronger workflow structure can increase the explicitness of intermediate artifacts. Third, stronger workflow structure can also impose substantial token overhead.

The pilot does *not* show workflow dominance, improved recoverability, or improved robustness. The tasks are too small, too clean, and too weakly perturbation-sensitive to test the paper’s main claims. This limitation is not incidental; it is central to correct interpretation. The pilot is included as a feasibility illustration, not as decisive evidence for the framework.

10. Hypotheses

The framework yields four falsifiable hypotheses.

H1. Workflow Dominance Hypothesis

On sufficiently long-horizon, tool-using tasks, workflow variation on a fixed model will often produce larger differences in practical performance than modest model variation under a fixed workflow.

H2. Recoverability Gap Hypothesis

Workflows with checkpointing and retry logic will outperform simpler workflows disproportionately under perturbation, even when clean-path quality is similar.

H3. Budget Divergence Hypothesis

Systems that appear similar in unconstrained quality will diverge substantially once token, latency, and retry budgets are enforced.

H4. Auditability Tradeoff Hypothesis

Some workflows will sacrifice a small amount of raw completion quality in exchange for materially greater trace legibility and repairability, making them preferable in oversight-sensitive settings.

These are hypotheses, not results. The mini pilot reported above does not adjudicate them.

11. Counterposition and Response

A strong objection to workflow-centered evaluation is that workflow variation is merely implementation variance. On this view, evaluation should standardize workflow as much as possible and compare models cleanly, because otherwise benchmarking loses comparability.

This objection is serious and partially correct. For short, self-contained tasks with little tool dependence and no persistent state, workflow standardization is often the right move. In such regimes, model comparison remains both interpretable and efficient.

However, for long-horizon, tool-using, budgeted workflows, treating workflow as nuisance variance risks standardizing away mechanisms that determine real-world usefulness. If retry discipline, checkpointing, trace persistence, or decomposition strategy materially affect recoverability, oversight, and operating cost, then excluding them from evaluation may increase comparability only by decreasing relevance.

The issue is therefore not whether workflow should ever be standardized. It is whether workflow should be standardized away even when it is part of the thing actual users depend on. In the target regime of this paper, the answer is often no.

12. Boundary Conditions and Threats to Validity

This framework has clear limits.

First, there are task regimes where model effects remain dominant. Short, self-contained reasoning tasks with minimal tool interaction are obvious examples.

Second, workflow-centered evaluation is likely to be more expensive, more bespoke, and more labor-intensive than leaderboard-style benchmarking. This is not a minor inconvenience; it is a real adoption constraint.

Third, environment effects may be difficult to separate cleanly from workflow effects. A workflow that appears superior in one tool ecosystem may simply be better adapted to local interface properties.

Fourth, the proposed metrics for recoverability and auditability remain provisional and partly evaluator-dependent. They are intended as tractable starting points rather than as closed-form solutions.

Fifth, the mini pilot reported here is intentionally tiny. It cannot adjudicate the paper's main empirical claim and should not be read as doing so.

Finally, there is a meta-risk of reconstituting benchmark theater at a higher level. A workflow-centered framework could itself become a new scoreboard culture if its dimensions are flattened too aggressively.

13. Discussion

The practical implication of this paper is that evaluation may need to become more infrastructural and less leaderboard-centric for a specific class of agent tasks. If the relevant object is a long-horizon workflow-conditioned system, then evaluation cannot remain solely a matter of ranking models by isolated benchmark scores. It must become a matter of protocol design, perturbation analysis, trace quality, and costed behavior under constraints.

This has several consequences. It weakens the assumption that there is a single scalar gap between systems; it elevates trace persistence and repairability from engineering conveniences to evaluation dimensions; and it suggests that orchestration, memory policy, retry logic, and budget discipline belong nearer the explanatory center of agent performance than benchmark culture usually allows.

More broadly, workflow-centered evaluation encourages a less theatrical conception of progress. It asks not simply which model scores highest, but which systems remain usable, recoverable, and inspectable under realistic constraints.

14. Conclusion

Benchmark culture remains useful, but it is no longer sufficient for serious evaluation of long-horizon, tool-using, budget-constrained agent workflows. In these settings, the relevant explanatory object is often the workflow-conditioned system rather than the base model alone.

This paper argued that observed performance should be analyzed as a function of model, workflow, environment, and budget. It defined five evaluation dimensions, operationalized recoverability and auditability as provisional proxy metrics, situated the proposal within recent benchmark and agent-method literature, and outlined both a factorized protocol and a small executable pilot.

The pilot did not show workflow dominance on easy tasks, and that negative result matters. It sharpens rather than weakens the scope of the argument: workflow-centered claims should be tested where recoverability, statefulness, tool use, and budget constraints genuinely matter.

The paper's strongest contribution is therefore not a demonstrated empirical result, but a falsifiable reframing of the evaluation problem. The framework would gain support if richer experiments show that workflow variation systematically changes performance, recoverability, or auditability under fixed-model

conditions. It would weaken if such effects remain small once tasks become realistic and perturbation-sensitive. That is the research program this paper proposes.

References

- Handa, D., Chintagunta, B., Brahman, F., et al. (2025). *ActionReasoningBench: Reasoning about Actions with and without Ramification Constraints*. ICLR 2025. arXiv:2406.04046. <https://arxiv.org/abs/2406.04046>
- Jimenez, C. E., Yang, J., Wettig, A., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. ICLR 2024. arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>
- Koh, J. Y., Lo, R., Jang, L., et al. (2024). *VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks*. ACL 2024. arXiv:2401.13649. <https://arxiv.org/abs/2401.13649>
- Liu, X., Yu, H., Zhang, H., et al. (2024). *AgentBench: Evaluating LLMs as Agents*. ICLR 2024. arXiv:2308.03688. <https://arxiv.org/abs/2308.03688>
- Mialon, G., Fourrier, C., Wolf, T., et al. (2023). *GAIA: A Benchmark for General AI Assistants*. arXiv:2311.12983. <https://arxiv.org/abs/2311.12983>
- Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., & Kambhampati, S. (2023). *PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change*. NeurIPS Datasets and Benchmarks. arXiv:2206.10498. <https://arxiv.org/abs/2206.10498>
- Wang, H., Deng, S., Zhao, S., et al. (2024). *Devil’s Advocate: Anticipatory Reflection for LLM Agents*. arXiv:2405.16334. <https://arxiv.org/abs/2405.16334>
- Xie, T., Zhang, D., Chen, J., et al. (2024). *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. arXiv:2404.07972. <https://arxiv.org/abs/2404.07972>
- Ye, C., Pan, H., Li, Y., et al. (2024). *MIRAI: Evaluating LLM Agents for Event Forecasting*. arXiv:2407.01231. <https://arxiv.org/abs/2407.01231>